

Student 3 **DANIMIZA V** 192421363RC

4 / 4 submitted

Q1. Intelligent Customer Feedback Analysis

CODE

```
positive_words = [" good","excellent","great","awsome","happy","love","best"]
negative_words = ["bad","poor","worst","hate","terrible","disappointed","slow"]

def preprocess(text):
    text = text.lower().strip()

    for ch in [".",",","!","?",":",";","'",'"',"(",")","-"]:
        text = text.replace(ch, "")
    return text

def analyze_feedback(text):
    words = text.split()
    positive_count = 0
    negative_count = 0

    for word in words:
        if word in positive_words:
            positive_count += 1
        elif word in negative_words:
            negative_count += 1
    if positive_count > negative_count:
        sentiment = "positive"
    elif negative_count > positive_count:
        sentiment = "negative"
    else:
        sentiment = "neutral"
    return positive_count, negative_count, sentiment

n = int(input("enter number of feedback entries:"))
for i in range(n):
    print("\nfeedback",i+1)
    feedback = input("enter feedback:")

    clean_text = preprocess(feedback)
    pos, neg, result = analyze_feedback(clean_text)

    print("processed feedback:",clean_text)
    print("positive keywords:",pos)
    print("negative keywords:",neg)
    print("sentiment:",result)
```

INPUT

```
2
excellent product! I love it.
bad service, very slow
The product is good but delivery was slow
```

OUTPUT

```
(no output)
```

Q2. Comprehensive Password Strength Checker**CODE**

```
def check_password(password):
    upper = 0
    lower = 0
    digit = 0
    special = 0
    for ch in password:
        if ch.isupper():
            upper += 1
        elif ch.islower():
            lower += 1
        elif ch.isdigit():
            digit += 1
        else:
            special += 1
    score = 0
    if len(password) >= 8:
        score += 1
    if upper > 0:
        score += 1
    if lower > 0:
        score += 1
    if digit > 0:
        score += 1
    if special > 0:
        score += 1
    if len(password) == 0:
        strength = "empty password"
    elif score <= 2:
        strength = "weak"
    elif score == 3:
        strength = "moderate"
    elif score == 4:
        strength = "strong"
    else:
        strength = "very strong"
    print("\npassword analysis:")
    print("length:", len(password))
    print("uppercase:", upper)
    print("lowercase:", lower)
    print("digits:", digit)
    print("special:", special)
    print("strength:", strength)
    print("\nfeedback:")
    if len(password) < 8:
        print("-password should contain atleast 8 characters.")
    if upper == 0:
        print("-add uppercase letters.")
    if lower == 0:
        print("-add lowercase letters.")
    if digit == 0:
        print("-add digits.")
    if special == 0:
        print("-add specail numbers.")
    password = input("enter password:")
    check_password(password)
```

INPUT

jeon@1998

OUTPUT

(no output)

Q3. Structured Server Log Error Analyser**CODE**

```

def extract_errors(logs):
    errors = []
    for line in logs:
        if "ERROR" in line:
            errors.append(line)
    return errors

def remove_duplicates(errors):
    unique = []
    for error in errors:
        if error not in unique:
            unique.append(error)
    return unique

def summarize(errors):
    error_count = {}
    for error in errors:
        if "404" in error:
            key = "404 error"
        elif "500" in error:
            key = "500 error"
        elif "timeout" in error:
            key = "timeout error"
        else:
            key = "other error"
        if key in error_count:
            error_count[key] += 1
        else:
            error_count[key] = 1
    print("\nunique error messages:")
    for error in errors:
        print(error)
    for key, value in error_count.items():
        print(key,":",value)

n = int(input("enter number of log entries:"))
logs = []
for i in range(n):
    log = input("enter log entry:")
    logs.append(log)

errors = extract_errors(logs)
unique_errors = remove_duplicates(errors)
summarize(unique_errors)

```

INPUT

```

6
INFO server started
ERROR 404 page not found
WARNING low memory
ERROR 500 internal server error
ERROR 404 page not found
ERROR timeout cancellation lost

```

OUTPUT

```

(no output)

```

Q4. Email Domain Frequency Analyser**CODE**

```
def extract_domains(emails):
    domains = []
    for email in emails:
        if "@" in email:
            domain = email.split("@")[1]
            domains.append(domain)
        else:
            print(email, "→ invalid email")
    return domains

def count_domains(domains):
    frequency = {}
    for domain in domains:
        if domain in frequency:
            frequency[domain] += 1
        else:
            frequency[domain] = 1
    print("\ndomain frequency:")
    for domain, count in frequency.items():
        print(domain,":",count)
    max_domain = ""
    max_count = 0
    for domain, count in frequency.items():
        if count > max_count:
            max_count = count
            max_domain = domain
    print("\nmmost used domains:",max_domain)
    print("frequency:",max_count)

n = int(input("enter number of email address:"))
emails = []
for i in range(n):
    email = input("enter email:")
    emails.append(email)
domains = extract_domains(emails)
count_domains(domains)
```

INPUT

```
5
john@gmail.com
jk@gmail.com
phwin@gmail.com
thee@gmail.com
bts@gmail.com
```

OUTPUT

(no output)